

bpcGit

A safety net and a history book for your SAP BPC content

In one sentence: bpcGit keeps every saved version of your BPC reports, input schedules, logic scripts, Data Manager files, security and BPF designs, so you can see what changed, who changed it, and put back any earlier version with a few clicks.

Something BPC has never had

Standard SAP BPC (NetWeaver) has no version history for its content. When someone saves a logic script, a report or a transformation file, the previous version is simply gone, unless somebody remembered to keep a copy by hand. Finding out what changed, or undoing a change that broke month-end, usually means detective work.

bpcGit brings proper version control to BPC, a capability BPC has never offered out of the box. It runs inside your SAP system, in the browser, and it is built for BPC people: you do not need to know Git or GitHub to use it.

No Git knowledge needed. You work in a normal BPC-style screen with buttons such as *Commit*, *History*, *Diff* and *Restore*. bpcGit does the Git work behind the scenes.

Git in plain words

Git is the tool software teams around the world use to keep the history of their work. GitHub and Bitbucket are websites that store that history safely. You only need a handful of words:

Word	What it means for you
Repository	The place where the history is kept, like a shared archive folder that never forgets. It lives on GitHub or Bitbucket.
Commit	Saving a snapshot of selected objects into the archive, with a short note such as "Fix FX rate in CLEAR_DATA script". Your name and the date are recorded automatically.
History	The list of all snapshots of an object: who, when and why.
Diff	A line-by-line comparison showing what is different between the saved version and BPC.
Restore	Putting a version from the archive back into BPC.
Branch	A separate line of work in the same archive, for example one for development and one for production.

A simple way to think about it

Think of the *Track changes* and *Version history* features of Word or SharePoint, but for everything in a BPC environment, and with a note on every change explaining why it was made.

What bpcGit looks after

Content	Examples
EPM workbooks	Reports and input schedules, company and team folders
Logic scripts	Script logic files (.LGF) of every model
Data Manager	Transformation and conversion files, packages and package links
Security	Teams, task profiles and data access profiles
Business process flows	BPF template designs
Dimension members	Members and their properties, per dimension

What you can do

1. See where everything stands

Choose an object type and a model, then **Load**. Every object gets a clear status, such as *Unchanged*, *Modified in BPC*, *New in BPC* or *Modified in Git*, so you can see at a glance what has changed since the last snapshot. Filters and a search box help in large environments.

2. Save a version (Commit)

Tick the objects you changed and choose **Commit**. Write a short note about the change. bpcGit saves all selected objects together as one snapshot, signed with your SAP user name.

3. See the history

Select an object and choose **History** to see every saved version: who saved it, when, and the note they wrote.

4. See exactly what changed (Diff)

For logic scripts, transformation and conversion files, Data Manager packages and package links, **Diff** compares the saved version with what is in BPC now, line by line: lines that were removed are shown in red, lines that were added in green.

5. Undo mistakes (Restore)

Put the saved version back into BPC, or go back to any older version from the history. bpcGit always shows what will be overwritten before anything happens, and files that are open for editing in BPC are left alone.

6. Move changes with transport requests

Add the selected objects to a customizing transport request with **Add to transport**, or record them automatically while restoring. bpcGit records them exactly the way BPC's own transport does, so the normal release and import process to quality and production systems works unchanged.

7. Large workbooks

Very large Excel workbooks can be stored with Git LFS (Bitbucket), which keeps the repository fast while every version stays available.

8. Nothing to install on your PC

bpcGit is a web application inside SAP. It works with GitHub and Bitbucket, and uses your normal SAP login.

Why it matters

Benefit	What it means in practice
A safety net	A broken script or report can be put back to its last good version in minutes, even during close.
A clear audit trail	Every change has a name, a date and a reason. Useful for auditors, for SOX-style controls and for anyone taking over the system.
Fewer surprises	Before you restore or move anything, you can see exactly what is different.
Better teamwork	Consultants, administrators and business users work from the same history instead of private copies and e-mail attachments.
Safer moves to production	Changes go through the normal SAP transport process, and the repository shows what each one contains.
Knowledge that stays	The notes on every commit explain why the system looks the way it does, long after the people who built it have moved on.
Disaster recovery	A copy of all BPC content lives outside SAP, on GitHub or Bitbucket.

A typical change, step by step

Step	What happens
1	An administrator updates the logic script that clears forecast data.
2	In bpcGit the script shows <i>Modified in BPC</i> . Diff confirms that only the intended lines changed.
3	They Commit it with the note "Clear forecast only for open periods".
4	They choose Add to transport and pick a customizing request, which is then released to production as usual.
5	If anything goes wrong, History and Restore put the previous version back in a few clicks.

The bottom line: bpcGit gives SAP BPC what modern software teams take for granted: a complete, searchable history of every change, and the confidence to change things because you can always go back.